



Distributed Systems

Fault Tolerance & Recovery



Recap

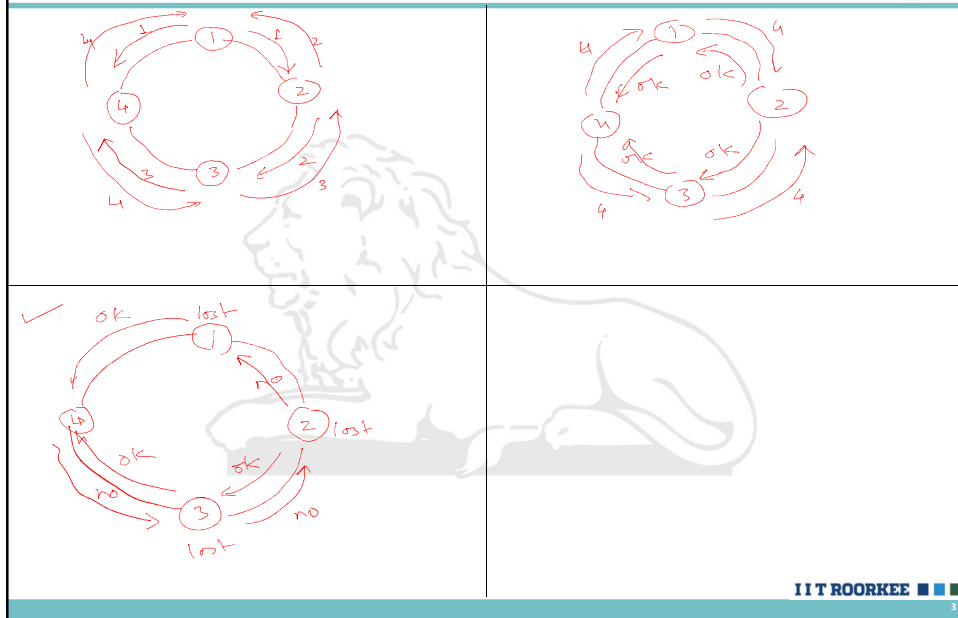


Elementary Graph Algorithms

- Spanning Tree
- Broadcast / Convergecast
- MST
- Leader Election



Hirschberg and Sinclair Algorithm (Example)



IIT ROORKEE

Fault Tolerance & Recovery

Classification of faults:

- **based on component that failed**
 - Process / processor / link / storage / clock
- **based on behavior of faulty component**
 - **Crash** – just halts
 - **Fail-stop** – crash with additional conditions
 - **Omission** – fails to perform some steps / messages lost
 - **Timing** – violates timing constraints
 - **Byzantine** – behaves arbitrarily

Types of tolerance:

- **Masking** – system always behaves as per specifications even in presence of faults
- **Non-masking** – system may violate specifications in presence of faults. Should at least behave in a well-defined manner

Fault tolerant system should specify:

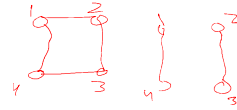
- Class of faults tolerated
- what tolerance is given from each class

Some building blocks (assumptions / primitives that help in designing fault tolerant systems):

- **Agreement** (multiple processes agree on some value)
(The Generals Paradox)
- Clock synchronization ✓
- Stable storage (data accessible after crash)
- Reliable communication (point-to-point, broadcast, multicast)
- **Atomic actions**

Agreement Problems

(Sync. Sys.)



Model:

- total n processes, at most m of which can be faulty
- **Assumption:** reliable communication medium
- fully connected (logically) - No network partition
- receiver always knows the identity of the sender of a message
- byzantine faults
- a faulty processor can refuse to send a message or may lie
- **synchronous system**. In each round, a process receives messages, performs computation, and sends messages.

The Problem

- All non-faulty processors must reach to an agreement between two values 0 and 1
- To reach an agreement processors have to exchange their values and relay the received values to other processors
- What a faulty processor can do?

Messages

- authenticated message (signed message)
- non-authenticated message (oral message)

Different problem variations

- **Byzantine agreement (or Byzantine Generals problem)**

- one process x broadcasts a value v
 - all non-faulty processes must agree on a common value (**Agreement condition**).
 - The agreed upon value must be v if x is non-faulty (**Validity condition**)

Note: If x is faulty then all non-faulty processors can agree on any common value

Different problem variations

- **Consensus**

- Each process broadcasts its initial value
 - satisfy agreement condition
 - If initial value of all non-faulty processes is v , then the agreed upon value must be v

Note: If initial values of non-faulty processes are different – they can agree on any common value

- **Interactive Consistency**

- Each process i broadcasts its own value v_i
 - all non-faulty processes agree on a common vector $(v_1, v_2, \dots, v_n)$
 - If the i^{th} process is non-faulty, then the i^{th} value in the vector agreed upon by non-faulty processes must be v_i

Solution to Byzantine agreement problem implies solution to other two

Agreement Problems

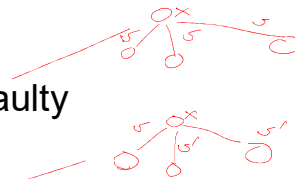
Problem	Byzantine Agreement	Consensus	Interactive Consistency
Who initiates the value	One Processor	All Processors	All Processors
Final Agreement	Single Value	Single Value	A vector of values

Byzantine agreement problem can be solved using a solution to the consensus problem.

1. The source processor sends its value to all other processors, including itself
2. All the processors, including the source, run an algorithm for the consensus problem using the value received in the first step as their initial values

(i) Source processor is non-faulty

(ii) Source processor is faulty



Byzantine Agreement Problem

- First defined and solved by Lamport et. al.
- no solution is possible if
 - asynchronous system, or
 - $n < (3m + 1)$ where **n** – total processors and **m** - **faulty** (synchronous system)
- needs at least $(m+1)$ rounds of message exchange (lower bound result)
- “Oral” messages – messages can be forged / changed in any manner, but the receiver always knows the sender
- Assumption: agreement is to be reached between 0 and 1

$$\begin{aligned} m &= 1 \\ 3m + 1 &= 4 \\ \boxed{n \geq 4} \end{aligned}$$

Byzantine generals algorithm



Boolean: $v \leftarrow$ initial value;

integer: $f \leftarrow$ max no. of malicious processes, $\leq \lfloor (n-1)/3 \rfloor$;

(message type) - OM(v , Dests, List, faulty), where

v - is a Boolean,

Dests - set of dest. process ids to which the message is sent,

List - list of process ids traversed by this message, ordered from most recent to earliest,

faulty - an integer indicating the number of malicious processes to be tolerated.

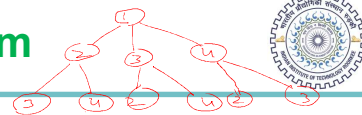
Oral_Msg (f), where $f > 0$:

- (1) Commander initiates and sends his source value v to all other processes using a OM(v , N , $\langle i \rangle$, f) message.

The commander returns his own value v and terminates.

IIT ROORKEE

Byzantine generals algorithm



(2) [Recursion unfolding]:

For each message of the form OM(v_j , Dests, List, f') received in this round from some process j , the process i uses the value v_j it receives from the source j , and using that value, acts as a new source. (If no value is recd, a default value is assumed.)

To act as a new source, the process i initiates Oral_Msg ($f' - 1$), wherein it sends

OM(v_j , Dests- $\{i\}$, concat($\langle i \rangle$, L), ($f' - 1$))

to destinations not in concat($\langle i \rangle$, L) in the next round.

IIT ROORKEE

Byzantine generals algorithm



(3) [Recursion folding]:

K+1 round

For each message of the form $OM(v_j, Dests, List, f')$ received in step 2, each process i has computed the agreement value v_k , for each k not in $List$ and $k \neq i$, corresponding to the value received from P_k after traversing the nodes in $List$, at one level lower in the recursion. If it receives no value in this round, it uses a default value.

Process i then uses the value majority _{$k \in List, k \neq i$} (v_j, v_k) as the agreement value and returns it to the next higher level in the recursive invocation.

IIT ROORKEE

17

Byzantine generals algorithm



Oral_Msg(0):

(1) [Recursion unfolding:] Process acts as a source and sends its value to each other process.

(2) [Recursion folding:] Each process uses the value it receives from the other sources, and uses that value as the agreement value. If no value is received, a default value is assumed.

IIT ROORKEE

18